

Plagiarism Detection Using Longest Common Subsequence and Jaccard Similarity

Harry Do, Justin Mei

Problem statement

Plagiarism detection is an important task in academic environments, online content platforms, and code repositories. Given two documents, determining the similarity can be computationally challenging, depending on the algorithm used. This brings us to the question: “How can we detect low-level plagiarism of copying text with minimal syntax changes?” Therefore, our project aims to compare two different approaches to document similarity detection: Longest Common Subsequence (LCS) and Jaccard Similarity

Algorithm description

1) Longest Common Subsequence (LCS)

The LCS similarity determines the longest subsequence of words in a given text with some other source material, and determines similarity by determining the ratio of the length of this subsequence to the total text length.

2) Jaccard Similarity

Jaccard Similarity is a set-based statistical similarity measure computed from the intersection and union of elements in sets. Jaccard is computationally faster and scalable, but less precise because it may ignore ordering and sentence structure. For instance, taking Figure 1 as an example:

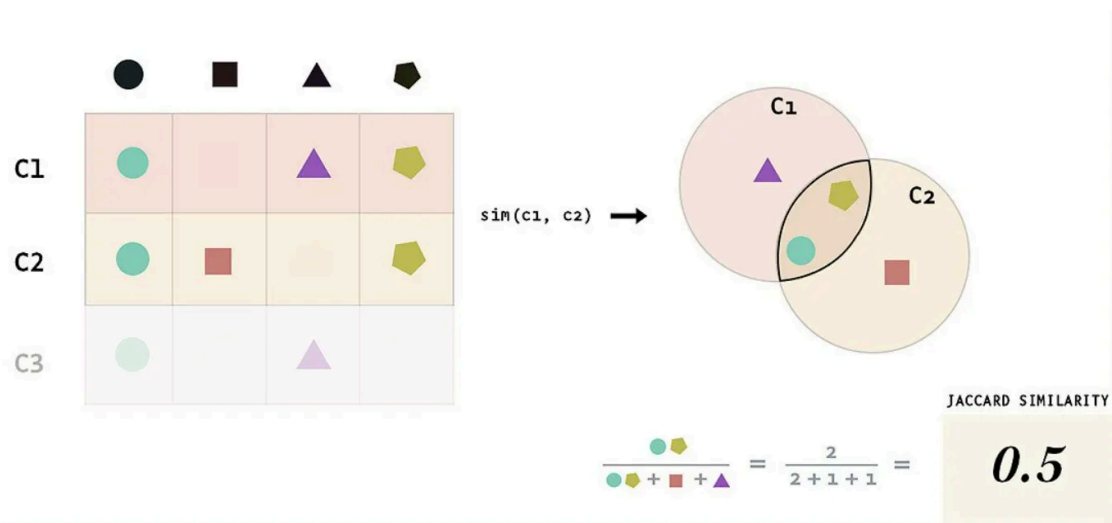


Figure 1

Illustration of Jaccard similarity between two sets.

The plagiarism detection algorithm is based on Jaccard Similarity, which measures the similarity between two sets of tokens, the algorithm will first preprocess all the input text by converting all character to lowercase and removing all punctuations, to make sure that the comparison are case-insensitive and unaffected by formatting differences. Then, the modified

texts will be splitted into individual words. In order to avoid bias from repeated words, each words is stored in an unordered set, this makes sure that every word is unique and duplicate tokens are only counted once.

Moreover, the Jaccard similarity is computed as the ratio of the intersection of the two token sets to the size of their union. It is defined in the following algorithm:

$$J(A,B) = (A \cap B) / |A \cup B| * 100, \text{ where}$$

$|A \cap B|$: the number of common elements between sets

$|A \cup B|$: the total number of unique elements in both sets

Furthermore, in order to compute overall similarity between two full paragraphs, the algorithm also divides each paragraph into non-overlapping chunks of up to 50 tokens and compute the Jaccard similarity of each chunks. This helps the algorithm to detect localized similarities and partial overlap between texts. Moreover, there are some exceptions might occur while using this approach for plagiarism detection. For instance, if both set are empty then the similarity is considered as 100% whereas if one set is empty, we will consider the similarity as 0%. Eventually, we will take the average of all chunks to consider the overall similarity.

Results and discussion

For Jaccard Similarity Approach:

```
File: sample.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog in a quiet field on a sunny aftern  on. This classic sentence is often used in typography and computer science to demonstrate how all letters of the English alphabet can appear within a single line of text. Because of its simplicity and balance, it is commonly chosen for testing fonts, keyboards, and text-processing systems.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat in a calm field during a warm aftern  on. This well-known sentence is frequently used in typography and computing to show how every letter of the English alphabet can be included in one short line of text. Due to its clarity and structure, it is widely selected for evaluating fonts, input devices, and text-analysis software.
Chunk 0: Jaccard = 41.6667%
Chunk 1: Jaccard = 28.5714%
Overall Jaccard = 35.8974%
Running time: 0.717 ms

File: sample_sent.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat.
Chunk 0: Jaccard = 54.545%
Overall Jaccard = 54.545%
Running time: 0.483 ms

File: testcase1.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat.
Chunk 0: Jaccard = 60.000%
Overall Jaccard = 60.000%
Running time: 0.240 ms

File: testcase2.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog in a quiet field.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat in a calm field.
Chunk 0: Jaccard = 60.000%
Overall Jaccard = 60.000%
Running time: 0.218 ms

File: testcase3.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog on a sunny aftern  on. This sentence is often used in typography to show how every letter of the English alphabet can appear in one line.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat during a warm aftern  on. This sentence is commonly used in typography to demonstrate how all letters of the English alphabet can fit into a single line.
Chunk 0: Jaccard = 50.000%
Overall Jaccard = 50.000%
Running time: 0.331 ms
```

```
File: testcase4.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog in a quiet field on a sunny aftern  on. This classic sentence is often used in typography and computer science to demonstrate how all letters of the English alphabet can appear within a single line of text.
--- Extracted Paragraph B (raw) ---
The quick brown fox jumps over the sleeping cat in a calm field during a warm aftern  on. This well-known sentence is frequently used in typography and computing to show how every letter of the English alphabet can be included in one short line of text.
Chunk 0: Jaccard = 43.636%
Overall Jaccard = 43.636%
Running time: 0.646 ms

File: testcase5.txt
--- Extracted Paragraph A (raw) ---
The quick brown fox jumps over the lazy dog in a quiet field on a sunny aftern  on. This sentence has become famous because it contains every letter of the English alphabet. As a result, it is widely used in typography, programming, and design to test fonts and text-rendering systems.
--- Extracted Paragraph B (raw) ---
On a warm aftern  on, the quick brown fox jumps over the sleeping cat in a calm field. The sentence is well known for including all letters of the English alphabet, which is why it is commonly used in programming, typography, and design to evaluate font styles and display systems.
Chunk 0: Jaccard = 40.000%
Overall Jaccard = 40.000%
Running time: 0.341 ms
```

The results show that Jaccard similarity approach effectively measures lexical overlap between pairs of texts. Noticeably, shorter inputs such as sample_sent.txt and testcase1.txt produce higher similarity scores (around 54-60%). This occurs because a large number of shared

words is contained in the total token set. On the other hand, longer paragraphs such as sample.txt and testcase4.txt hold lower similarity values (around 35-44%). The reason behind this fluctuation is the appearance of more unique tokens, which increases the union size and reduces overall similarity ratio.

Moreover, from a performance perspective, the algorithm runs in near-linear time, with an expected time complexity of $O(n+m)$, where n and m are the numbers of tokens in the two paragraphs. In addition, the algorithm stores tokens and unordered sets for both paragraphs, which is beneficial for the memory usage, resulting in a space complexity of $O(n+m)$.

While an algorithm using Jaccard similarity is effective for determining similar word choices, this could lead to false positives when writers use jargon specific to niche fields. For cases where entire sentences are copied, with either no modification or simple ones of rearranging words or using synonyms, we can utilize the LCS algorithm. Determining the ratio of the longest substring to the source material length can reveal how much text is copied and determine if plagiarism was committed. A naive implementation of LCS could run into the same issues as Jaccard if the two texts are compared in their entirety, as one sentence being completely copied could be buried if the texts are of sufficient length. We add a parameter to our similarity algorithm to choose how many sentences to compare at a time to allow for edge cases such as this one.

```
similarity(a, b) << '\n'; 0.235294
similarity(a, b, 1) << '\n'; 0.764706
similarity(a, b, 2) << '\n'; 0.705882
```

However, we must choose this parameter by balancing computational costs with algorithm accuracy. We can observe that comparing entire texts results in relatively low similarity despite extremely similar text, while there is little difference between the outputs, despite theoretically doubling the computational time when considering one and two sentences at a time. The time and space complexities of LCS are $O(m \times n)$ for lengths m and n , but m will typically not be the full length due to the sentences parameter.



The code uses a dynamic programming implementation of LCS, resulting in the $O(m \times n)$ complexity. Raw text is parsed into an array of sentences, determined by period (or other punctuation) delimiters, and further parsed into arrays of words representing strings. While using a sentence parameter of k , slices from the first text of k sentences are flattened into one sentence array to be considered against the second text. We then take the maximum length similarity sequence and divide it by the first sentence length to get a similarity score. A practical implementation would use several values of k , while maintaining an overall $O(m \times n)$ time/space complexity.

With both algorithms in mind, we can consider both outputs to better evaluate plagiarism as they cover each other's weaknesses. An LCS score of nearly 1.0 with a relatively low sentence count is an indicator of likely plagiarism, likewise with a Jaccard score of 0.5+. In such cases of copying, at least one score will be sufficiently high to indicate possible use of source material.